# E.C.H.O: An Intent Aware Personal Memory System for Unified Search Across Email, Web, and Video History

1st Arjun Sai N
*Dept. of Computer Science and Engineering*
*PES University*
Bengaluru, India

2nd Mohammed Mir Fazlai Ali
*Dept. of Computer Science and Engineering*
*PES University*
Bengaluru, India

3rd Mohammed Shehzaad Khan
*Dept. of Computer Science and Engineering*
*PES University*
Bengaluru, India

4th Mrigank Jain
*Dept. of Computer Science and Engineering*
*PES University*
Bengaluru, India

5th Mannuru Shalima Sulthana
*Dept. of Computer Science and Engineering*
*PES University*
Bengaluru, India

***Abstract*—In a single day, a person meets information in many places: the inbox, the browser, and video sites. Each place keeps its own history behind its own search box, so finding something again starts with guessing where it was seen. We present E.C.H.O (Efficient Contextualized History Orchestrator), a personal memory for a single user. It collects Gmail, Chrome, and YouTube activity into one store on the user's own laptop and answers questions about it in plain language. All mail is kept, except what Gmail itself files as spam or trash. A web page or a video is kept only when signals such as foreground time, scrolling, watch time, or a return visit show that the person actually attended to it. In the background, a worker cleans each item, pulls text out of its attachments, assigns one of five broad categories, and embeds it for search by meaning. A question goes to a language model that may call a single search tool. That tool is plain code. It runs keyword and vector search together, fuses the two ranked lists by reciprocal rank fusion, widens the search when the results look weak, and reorders the survivors with a local cross encoder. A typical question costs two model calls, and the code refuses a seventh. Storage, indexing, and search all run locally, and the model receives the question and short excerpts, never whole emails, pages, or transcripts. A wellbeing module describes how time is spent without judging it, and it sorts items into personal goals only with the user's approval. This paper explains the design and the evaluation protocol, and the measured results will follow.**

***Index Terms*—personal information management, retrieval augmented generation, hybrid search, reciprocal rank fusion, tool calling agents, intent aware capture, local first software, digital wellbeing**

## I. Introduction

Suppose a student is preparing for a job interview. The company sends an email, and over the following days the student reads articles about operating systems in Chrome and watches a few tutorials on YouTube. A week later, they want that material back. Now the email sits in Gmail, the articles are buried in the browser history, and the videos are in the YouTube watch list. That is three apps and three search boxes, and none of them knows about the other two. So the student either searches all three or starts again from scratch.

The idea of a machine that remembers what its owner has seen is not new. Bush called it the memex in 1945 [1], and later systems such as Stuff I've Seen [2] and MyLifeBits [3] built personal stores of mail, pages, and documents. But two problems still stand in the way of daily use. The first is noise. A browser logs every page that opens, including stray clicks and tabs closed after two seconds, so a memory built on that log fills up with things nobody read. The second is recall by exact words. A search box needs the right keyword, while people remember loosely, as in "the OS material I read after the interview email."

E.C.H.O addresses both problems. It watches Gmail, Chrome, and YouTube, but it keeps a page or a video only when engagement shows real attention. All items share one data model on the user's laptop. They are enriched in the background and can be queried in natural language. Following retrieval augmented generation [4], every answer is grounded in the stored items.

The main contributions are:

- An intent aware capture layer with one rule set per source, together with privacy filters that act before anything is stored. Sensitive pages lose their text, secret parameters are cut from addresses, and private work apps contribute only a page title.

- A local storage model in which emails, pages, and videos share one central table. PostgreSQL is the permanent record, the FAISS index can be rebuilt from it, and Redis holds short lived state. Attachment text is extracted when the mail arrives, so it becomes searchable too.
- A bounded agentic retrieval loop. The language model plans and phrases the searches, while a deterministic tool does the actual searching, meaning hybrid keyword and vector search, rank fusion, a rule based quality check with widening, and cross encoder reranking. Two model calls are typical, and six is a hard limit in the code.
- A wellbeing module that reports time, sessions, and self declared regret in descriptive terms only. Alongside it, a grouping engine never files an item under a personal goal without the user's explicit approval.

Section II reviews related work, and Section III gives an overview of the system. Section IV covers capture, storage, enrichment, and retrieval. Section V describes the wellbeing module and user groups. Section VI covers privacy, Section VII the implementation, and Section VIII the evaluation protocol. Finally, Section IX lists the limitations and Section X concludes.

## II. Related Work

### A. Personal Information Management

Bush imagined the memex as a desk that holds a person's books, records, and letters and brings any of them back quickly [1]. Stuff I've Seen built one index over everything a person had already seen, whether an email, a web page, or a document, and used that prior exposure as a cue for search [2]. MyLifeBits kept personal data in a SQL database, with the aim of storing everything that could be captured [3]. E.C.H.O shares the same goal of refinding, but it differs in two ways. First, it filters at capture time instead of storing everything. Second, it answers questions in natural language instead of offering only a query box.

### B. Retrieval Augmented Generation and Tool Use

Retrieval augmented generation couples a retriever with a text generator so that the generated answer draws on the retrieved documents [4]. E.C.H.O applies this idea to a personal collection and arranges the steps as a graph built with LangGraph [5]. ReAct showed that a language model can interleave reasoning with actions such as search calls, and that grounding each step in tool output reduces invented facts [6]. We borrow this pattern, but in a deliberately narrow form. The model has exactly one read only tool, the tool contains no model of its own apart from two small local encoders, and the number of rounds is capped in the code.

### C. Hybrid Search and Reranking

Sentence embeddings such as Sentence BERT make it possible to compare texts by meaning, using cosine similarity [7]. E.C.H.O embeds every item with a small local sentence model and searches the vectors with FAISS [8]. Keyword search uses the full text search built into PostgreSQL. The problem is that the two methods score on unrelated scales, so simply adding their scores is not meaningful. Reciprocal rank fusion avoids this by using only the position of each item in each list [9]. After fusion, a cross encoder reads the question and a candidate together and scores how well they match. Nogueira and Cho showed that this works well for passage reranking [10]. Our cross encoder is a compact MiniLM [11] model trained on the MS MARCO passage ranking data [12].

### D. Local First Software

Kleppmann et al. describe local first software, in which users keep ownership of their data and the application works from a copy on their own device [13]. E.C.H.O follows the same idea. The database, the vector index, the cache, and both encoders run on the user's laptop, and only small, chosen pieces of text are sent to a hosted language model.

### E. Digital Wellbeing Tools

Lyngs et al. reviewed hundreds of digital self control apps and browser extensions and found that most of them block, limit, or track usage against a goal [14]. E.C.H.O takes a quieter position. It shows where the time went and lets the user mark the items they regret. But it never scores the user, never blocks content, and never labels time as wasted.

## III. System Overview

Fig. 1 shows the main parts of the system. Data comes in through three connectors. The Gmail connector reads mail through the Gmail API with OAuth, and it asks only for the changes since its last check. A Chrome extension observes desktop browsing and YouTube playback, and the YouTube connector fills in the video details from the YouTube Data API. Every connector hands its items to one FastAPI gateway. Its storage engine writes each item in a single transaction and skips duplicates.

Each of the three stores has one job. PostgreSQL is the permanent record of every item. FAISS holds the embedding vectors, and it can be rebuilt from PostgreSQL. Redis keeps the short lived state, meaning revisit flags, cached text, rate limits, and the user's capture settings. All three run on the user's laptop.

Three modules then work on the stored data. The enrichment worker prepares new items for search in the background. The retrieval engine answers questions. The wellbeing module turns the stored activity into time summaries, sessions, regret patterns, and group suggestions. The main interface is a React dashboard, and a small extension popup lets the user pause capture.

## IV. Methodology

### A. Intent Aware Capture

What should E.C.H.O keep? Each source has its own answer, summarized in Table I. The values are the ones chosen for the final system. The browser and the server hold matching copies of them, so the server never silently refuses a page that the extension sends.

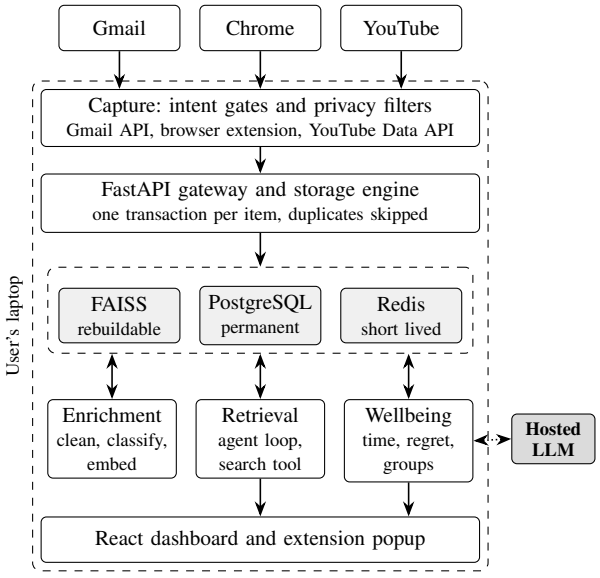



Fig. 1. Main parts of E.C.H.O. Everything inside the dashed frame runs on the user's laptop. The dotted arrow is the only path to the hosted language model; it carries short text from three callers (classification, retrieval, weekly reflection), described in Section VI.

Gmail is treated differently from the other two sources. Mail arrives on its own instead of being chosen by the user, the daily volume is small, and even an unread message may matter later. So every message is stored when it arrives. The connector polls every 30 seconds, but each poll asks Gmail only for the changes since a saved history position, instead of listing the entire mailbox. If that position has expired, the connector lists the mailbox once, and the uniqueness rule on stored items absorbs the overlap. Mail that Gmail has filed as spam or trash is never read, so Gmail's own spam filter acts as the spam filter of E.C.H.O. When the user opens a message in Gmail on the web, the extension reports the message id and the seconds spent reading, and nothing else.

A Chrome page passes a two phase gate. In phase one, the page must stay in the foreground for at least five seconds. A page that fails is dropped, and only a page that passes is remembered for revisit detection. In phase two, at least one sign of attention must appear: enough foreground time, enough scrolling, a real interaction, or a visit to the same page within the last 24 hours. Selecting text or clicking inside the page counts as an interaction, but clicking a link does not. The reason is that a click that leaves the page shows a choice to go elsewhere, not interest in the page itself. Time is counted only while the tab is active, the window has focus, and the user has not been idle for 30 seconds. Along with the text of the main content, the extension sends the top level headings of the page, which later help the page to be found by its topic.

A YouTube video is kept when the user watches it long enough, watches a large enough share of it, pauses, seeks, or changes the speed, or returns to the same video within a day. The completion rule requires at least five seconds of real watching, so swiping past a short clip does not count as watching ten percent of it. Once a video has been watched for five seconds, the extension records the visit, so that a second short visit can pass through the revisit rule. Shorts are recognized from the page address, and they have a lower time bar.

TABLE I
CAPTURE RULES FOR EACH SOURCE

| Source | An item is kept when |
|---|---|
| Gmail | Always, at arrival, except mail in Spam or Trash. Polled every 30 s for changes only. Reading time is added when a message is opened (opens under 3 s are ignored). |
| Chrome | Phase 1: at least 5 s in the foreground. Phase 2: any one of at least 10 s in the foreground, at least 15% scroll depth, one interaction other than a link click, or a revisit within 24 hours. Work app pages: at least 5 s or one interaction, title only. |
| YouTube | Any one of at least 20 s of foreground playback (10 s for Shorts), at least 10% of the video watched with at least 5 s of watching, a pause, seek, or speed change by the user, or a revisit within 24 hours. |

### B. Storage Model

Each email, page, and video becomes one row in a central table of memory items. Fields that belong to only one source go into a matching table for Gmail, Chrome, or YouTube, and attachments get a table of their own. Engagement counters such as reading time, watch time, and revisits are kept separately, because they change often. Chrome and YouTube also write one row for every admitted visit, with the time of the visit and the seconds spent. This way, a page read on two different days is counted on both days. In total, the schema has 17 tables.

A uniqueness rule on the pair of source type and source id stops the same item from being stored twice, for example after a connector restarts. Every item always carries one of the five system groups, with misc as the fallback. Items are first written in a raw state and flagged as unprocessed, which keeps the capture path short and safe against crashes. If a revisited page comes back with different text, it is flagged again so that its vector is rebuilt. Timestamps are stored in UTC and shown to the user in local time.

A user can delete a single item, which hides it from every query. Separately, a retention setting offers three choices: keep forever, keep one year, or keep six months. Keep forever is the default, because recalling old things is the whole point of the system. If the user picks a shorter window, a job at backend startup permanently removes the older items from PostgreSQL, the FAISS index, and Redis together.

### C. Background Enrichment

A background worker picks up the unprocessed items every few seconds. It never runs on the capture path, so a slow enrichment step cannot slow down capture. Each item goes through four steps.

*Cleaning.* For email, HTML is converted to text with the line breaks kept, and scripts, styles, tracking pixels, and hidden

preview text are removed. When the sending app has split a body around an inline image, the parts are joined back together. Quoted earlier messages are removed, but the lines the sender typed between quoted lines are kept, since an inline reply is often the whole point of the message. Signatures are deliberately left in place. A heuristic remover we tested cut real body text in six of eight sample messages, while a leftover signature only adds a little noise. For a reply, keywords from earlier messages in the same thread are added, so that a terse "Re:" can still be found by its topic. For web pages, the extension has already picked the main content and the headings, and the worker separates the headings from the body. For videos, the text is made of the title, the channel, the category, the tags, the description, and the transcript. The server fetches the transcript with a transcript library, trying English captions first and automatic captions second. A video without captions is embedded without one.

*Attachments.* The same worker extracts attachment text when the mail is processed. The supported types are PDF, DOCX, XLSX, PPTX, plain text formats such as TXT, CSV, Markdown, JSON, and calendar files, HTML, and forwarded messages. Images and scanned PDFs are read with Tesseract OCR [15]. Inline images such as signature logos, and very small images, are skipped. The extracted text is stored next to the attachment record, not inside the email body. The beginning of each attachment joins the embedded text of the email, while the full text joins the keyword index. So content deep inside a file can still be found by keyword.

*Embedding.* The cleaned text is turned into a 384 dimension vector with the `granite-embedding-small-english-r2` sentence model **[TBD: confirm final model, decision A40]**. The text is assembled such that the short hints come first. For mail these are the subject, keywords, and sender, for pages the domain and headings, and for videos the channel and category. Long text such as a body or a description comes last. So if the text has to be trimmed to the input window of the model, only the end of the long part is removed **[TBD: token budget, decision A16]**. The vector is added to the FAISS index without rebuilding the index.

*Classification.* Every item receives one of five broad groups, which are work, study, entertainment, personal, and misc. The group is chosen by a four stage cascade. Stage one reads structural signals from the source, such as Gmail category labels, academic mail domains, or the YouTube video category. Stage two looks up the web domain in a curated table. Stage three compares the vector of the item with a center point for each group and accepts only a clear winner. Each center starts as the mean of several example sentences written for its group, and while it is still a seed it uses a lower acceptance bar **[TBD: seed threshold, measured on labeled rows]**. Once a group has at least ten confirmed items, its center is recomputed from real items. Each source is weighted equally in this step, so that a center learns a topic and not the look of one source. A recomputed center accepts an item only if the cosine similarity is at least 0.55 and leads the second best group by at least 0.08. The recomputation runs when the worker starts, if the stored centers are missing or older than 30 days. Stage four asks the language model, one item at a time, using only the title and about 300 characters of text. With an OpenAI model, the call is a typed choice among the five group names. With other providers, the reply must be exactly one group name, and any other reply becomes misc. If the typed call raises an error, that item takes the single word path instead, so classification never stops. No confidence value is stored.

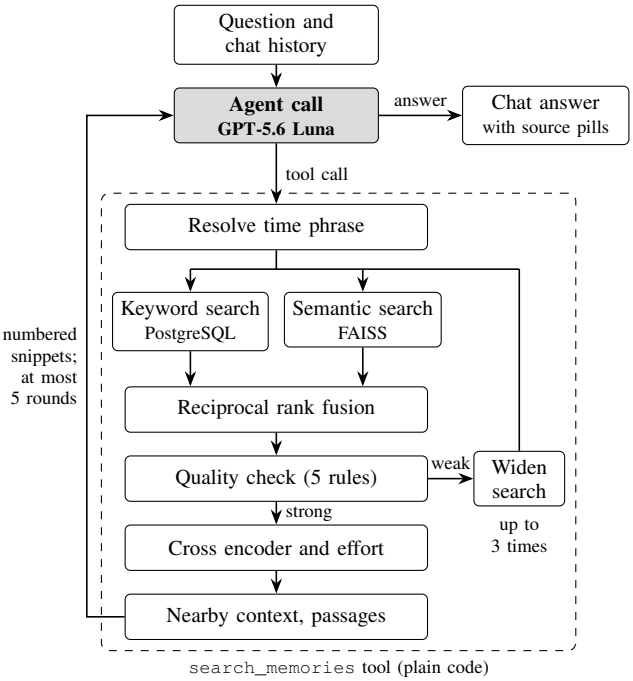



Fig. 2. The retrieval loop. The shaded box is the only place a language model is called. Everything inside the dashed frame is deterministic code with two small local encoders. When the search is widened, both search branches run again.

*State tracking.* Each item ends as either processed or failed. Failed items are kept apart and retried automatically, so a failure is never counted as done, and it is never lost.

### D. Retrieval Loop

How is a question answered? It passes through the loop in Fig. 2. The language model, GPT-5.6 Luna, receives the conversation so far, today's date, the question, and one tool called `search_memories`. The tool takes a query, an optional list of sources, an optional time phrase, and an optional sender or domain. The model decides how to phrase the search, and it may issue several searches in parallel in one round, for instance one per source. After reading the results, it either answers or searches again. A usual question needs one call that searches and one call that answers. The code allows at most five search rounds. After the fifth round, the next call is made without the tool, which forces an answer. So a question costs between two and six model calls, and this bound is enforced by the program, not by the prompt.

*Resolve time phrase.* The model passes the time words exactly as written, and the code turns them into dates. A

calendar phrase such as "last Tuesday" becomes a start date. A phrase that points at another item, such as "after the interview email," is looked up among the user's own items. The lookup goes first by keyword and then by meaning, with a minimum similarity. The time of the matching item then becomes a hard limit for both search branches. If no anchor item is found, the search runs without that limit. Mail is matched by when it arrived, while pages and videos match when any of their recorded visits falls in the range. The date filter is open ended, so the answering call is told today's date and asked to use only the items inside the period the user named.

*Parallel search.* The keyword branch runs the PostgreSQL full text search over a stored text index. This index covers titles, the full item text, attachment text, senders, domains, and channel names. The semantic branch embeds the query and searches FAISS. Both branches run in the same step of the graph.

*Fusion.* The two ranked lists are merged with reciprocal rank fusion [9]. For an item $d$, the fused score is

$$\mathrm{RRF}(d) = \sum_{b \in B} \frac{1}{k + r_b(d)}, \tag{1}$$

where $B$ is the set of branches that returned $d$, $r_b(d)$ is its position in branch $b$ starting from 1, and $k = 60$. Since only the positions are used, the two score scales never have to be matched. The top 50 fused items move on to the next step.

*Quality check and widening.* Five rules judge the fused list. The list is weak or empty if it has no items, if no item comes from a source the user asked for, if no item falls inside the time window, if it holds fewer than two items, or if the best cosine similarity is below 0.35. A weak list triggers a wider search, and both branches run again. Each attempt relaxes more than the one before. The first attempt stretches the time window by four days, opens all sources, or shortens the query to its core keyword. The second drops the time filter and the anchor. The third drops the keyword filters and searches a larger part of the vector index. After three attempts, the tool reports that nothing was found, and the model can either say so or try a different phrasing.

*Reranking.* The local cross encoder `ms-marco-MiniLM-L-6-v2` [10], [12] scores each remaining item against the question. The final score blends this with an effort score:

$$s(d) = 0.8\,\sigma\big(c(q,d)\big) + 0.2\,e(d), \tag{2}$$

where $c(q,d)$ is the cross encoder output for question $q$ and item $d$, $\sigma$ is the sigmoid function, and $e(d) \in [0,1]$ is the effort score. Effort is the mean of the engagement signals that an item has, such as reading or watch time, scroll depth, revisits, interactions, and the share of a video watched. Each signal is capped and scaled to $[0,1]$. This favors the items the user gave real attention to. Email is a special case, because mail read on a phone, or never opened in the browser, has no measurable reading time. Instead of scoring such mail as zero, the reranker gives it the median effort of the other emails in the same result list. This way, an unmeasured email is neither punished nor promoted. By design, the score has no term for how recent an item is, and no diversity reranking is applied.

*Nearby context and passages.* For the top three items, the tool adds their close neighbors, meaning other emails in the same thread, or pages and videos from the same browsing session or within five minutes. Each result reaches the model as a numbered entry. The entry holds its title, source, time, the sender, site, or channel, and a 500 character window of its cleaned text. This window is the part that best matches the question, not simply the first 500 characters. Suppose a result is an email with attachments and the question is about their content. Then the three paragraphs of the stored attachment text that best match the question are added, about 1,500 characters in total. The numbers stay the same across rounds, so a citation always points to the same item.

*Writing the answer.* The final call writes a conversational reply in Markdown. The reply leads with the direct answer, admits gaps plainly, and asks back in one line when the question is ambiguous. After each fact, it places a hidden marker that names the numbered item supporting it. The backend keeps only the markers that point to items the tool actually returned, so the model cannot cite a source it never saw. The dashboard turns each marker into a small pill. The pill names the human anchor of the memory, such as the sender, the site, or the channel, along with a short date, and clicking it opens the original email, page, or video. A thin time strip under the reply places each cited item on a timeline. While the loop runs, the dashboard streams progress messages, such as which source is being searched, and then it streams the reply itself.

*Failure handling.* If the model cannot be reached, the deterministic pipeline runs once on the raw question. The user then receives the ranked list of matching items instead of an error.

### E. Conversation Memory

Each chat has a session id, and every turn is saved in PostgreSQL on the laptop. When the user opens an old chat, its session resumes, so a follow up such as "show me Chrome pages about that topic after that email" can refer back to an earlier answer. Chat history older than 30 days is removed at startup.

## V. Wellbeing and User Groups

The wellbeing module answers one question: where did my time go? Its rule is to describe and never to judge. It shows the numbers and patterns, and it leaves the verdict to the user.

*Time.* Time totals are computed on demand from the stored records and are never stored separately, which avoids counting the same minutes twice. Page and video time come from the per visit rows, so each day receives exactly the time spent on that day. Email time comes from the reading time that the extension reports. All days are the local days of the user.

*Sessions.* A session is a run of activity in which consecutive items are less than 20 minutes apart. The architecture first proposed five minutes. But each item is recorded at one point

in time, so a 15 minute read followed by the next page looked like a 15 minute gap, and one sitting was split in two. For each session, the module records its length, the system group with the most time, and how often the user switched between sources.

*Regret.* Only the user decides what is regrettable. Any item can be marked, and every mark or unmark is stored as a new event. So the record is never overwritten, and an item is currently regretted when its count of events is odd. The module reports how many marked items fall in each content type and hour. It flags a habit when the same kind of item, at a similar hour, is marked at least three times in 30 days.

*Weekly reflection.* Once a week, the module can write a short reflection. One language model call receives only aggregated numbers: totals per category, regret counts, session patterns, and peak hours. It never receives a title, an address, or any text from an item. Before the reply is shown, it is checked for judging phrases. If it fails this check, or if the call itself fails, a plain summary built from the same numbers is shown instead.

*User groups.* System groups answer "what kind of item is this?" User groups answer "which of my goals does it serve?", for example "Placement prep." A user group starts with the items the user files by hand. Once it has five confirmed items, an assignment job scores other items against it in two ways. The first is the rules the user wrote, each on a keyword, domain, channel, sender, or date range. The second is a nearest neighbor score, meaning the mean cosine similarity between the item and the three most similar confirmed members. An item is suggested when a rule matches, or when the neighbor score alone reaches a threshold **[TBD: KNN only threshold, decision A32, set with the final model]**. The module can also propose a new rule when at least five confirmed members share a domain, sender, or channel.

Every suggestion waits for the user. The only way an item enters a user group is an explicit accept in the review screen. Manual filing passes through the same recorded path, so each membership has an audit trail. A rejected suggestion is never offered again for that group. User groups are independent of the five system groups, so one goal can hold study pages and work mail together.

## VI. Privacy by Design

*What is never captured.* Nothing from incognito windows is recorded, and neither is typed text or form input. Some pages keep no text at all. This is the case when the address contains a sensitive word, matched as a whole word against a fixed list of 38, such as login, password, checkout, otp, or netbanking. The same holds for a page with a visible password box, or a form with several personal fields. Parameters that often carry secrets, such as tokens, one time codes, and session ids, are cut from every address before it is stored. Seven work apps are stored as page titles only and never as text: Slack, Atlassian, Notion, Figma, Linear, Trello, and Asana. Their public help, blog, and documentation sections are treated as ordinary pages. Gmail on the web is skipped by browser capture, because the mail already arrives through the Gmail API. The lists live in one copy on the server, and the extension fetches them at startup. Until they arrive, the extension extracts no text.

*What leaves the laptop.* The database, the vector index, the cache, and both encoders run locally. Two kinds of traffic leave the machine. Requests to the Gmail and YouTube APIs fetch the user's own data. Calls to the hosted language model carry short text from three callers. Retrieval sends the question, the chat history, and the numbered results. Each result is a title, a time, a sender, site, or channel, and a 500 character excerpt. Retrieval also adds at most three short passages of attachment text when the user asks about a file. The classifier sends a title and about 300 characters, and it does so only for items that the first three stages could not place. The weekly reflection sends only aggregated numbers. Full emails, full pages, transcripts, database rows, and vectors are never sent.

*Captured text is data, not instructions.* A stored email or page might contain text that was written to steer a language model. The search tool is read only, and the system prompt tells the model that every snippet is data. Together, these limit what such text can achieve. Retrieval logs record counts, scores, and timings, but not questions or titles.

*User control.* Each source can be switched off, domains and senders can be excluded, and capture can be paused from the extension popup. Items can be deleted, all the data can be exported as JSON or CSV, and the retention setting removes old data for good.

## VII. Implementation

Table II lists the technology used. The backend is written in Python and served by FastAPI. The retrieval loop is a LangGraph graph [5] in which an agent node and a tool node alternate under a round counter. Every model call goes through LangChain, so switching to a different provider means changing one configuration entry. When the provider is OpenAI, the code also turns on features specific to that provider: the Responses API, a medium reasoning effort for both retrieval calls, and the typed choice used by the classifier. The embeddings and the cross encoder run locally on the CPU through the sentence transformers library. The dashboard is built with React and TypeScript, and the extension uses Chrome Manifest V3.

PostgreSQL 16 runs as a native service on the laptop, and Redis runs locally under the Windows Subsystem for Linux. During development we used hosted database and cache services. The final system moved both of them to the laptop, which also means that only standard HTTPS traffic leaves the machine.

## VIII. Evaluation Protocol

This draft reports no measured results yet. Every value below that is marked TBD will be filled in from measurements on the final build.

TABLE II
TECHNOLOGY USED IN THE FINAL SYSTEM

| Layer | Technology |
|---|---|
| Capture | Chrome extension (Manifest V3), Gmail API, YouTube Data API |
| Backend | Python, FastAPI, server sent events |
| Storage | PostgreSQL 16, FAISS, Redis, all local |
| Enrichment | Text extraction for PDF, DOCX, XLSX, PPTX, and EML; Tesseract OCR |
| Models | `granite-embedding-small-english-r2` embeddings, `ms-marco-MiniLM-L-6-v2` cross encoder, GPT-5.6 Luna through the OpenAI Responses API |
| Retrieval | LangGraph, LangChain |
| Interface | React, TypeScript, Vite |

TABLE III
RETRIEVAL QUALITY (TO BE MEASURED)

| Configuration | Recall@10 | MRR |
|---|---|---|
| Keyword only | **[TBD]** | **[TBD]** |
| Semantic only | **[TBD]** | **[TBD]** |
| Fusion, no cross encoder | **[TBD]** | **[TBD]** |
| Full pipeline | **[TBD]** | **[TBD]** |
| Full agent loop | **[TBD]** | **[TBD]** |

### A. Data and Questions

The evaluation uses the test account of the project, restored into the local database: **[TBD: number of items per source]**. The team will write **[TBD: number of questions]** questions, each with known relevant items. These cover five kinds of need: a single source, several sources, a time anchored question ("after the interview email"), a follow up that refers to an earlier turn, and a question about attachment content.

### B. Retrieval Quality

We compare four configurations: keyword search alone, semantic search alone, fusion without the cross encoder, and the full pipeline. For each, we report Recall@10 and the mean reciprocal rank of the first relevant item in Table III. The same question set is also run through the full agent loop. This shows whether letting the model rephrase and repeat its searches finds items that the single pass misses.

### C. Cost and Speed

For each question, we log the number of model calls, the number of search rounds, the input and output tokens, and the end to end time on the team's laptop. From these logs we will report the mean number of calls **[TBD: mean LLM calls per question]**, how often the five round cap was reached **[TBD: share of questions hitting the cap]**, the median and 95th percentile response time **[TBD: latency, seconds]**, and the cost per question **[TBD: cost per question]**.

### D. Capture Accuracy

We replay scripted browsing and viewing sessions, such as a page closed after two seconds, a page scrolled but left quickly, a Short swiped past, and a tutorial watched halfway. The stored items are then compared with what each rule should keep **[TBD: agreement of the gates with the scripted labels]**.

### E. Privacy Audit

During the evaluation, every request to the language model is logged. We check that no request contains a full email, page, or transcript, and we report the largest request seen **[TBD: largest request, characters]**.

### F. Threshold Calibration

Three similarity cutoffs depend on the embedding model: the seed acceptance bar of classification stage three, the nearest neighbor bar for group suggestions, and the 0.35 floor of the quality check. So they will be set on the restored data once the model is fixed. For the first two, we pick the lowest value that produces no wrong assignment on the items with known labels.

## IX. LIMITATIONS AND FUTURE WORK

E.C.H.O has clear limits. It targets desktop Chrome, so activity on the phone is invisible. YouTube history starts on the day the extension is installed, and background or picture in picture playback is not tracked. Since each item has one vector, only the beginning of a long document shapes its meaning based match, and deep content can be reached by keyword only. Replies from Outlook quote the older thread without a marker that E.C.H.O removes, so such threads can appear more than once in the results. The date filter is open ended, and only the top ten results reach the answering call. So on a busy day, a question about yesterday could be crowded out by the items of today. Pages of private apps outside the seven listed, such as chat assistants or online documents, are captured as ordinary pages unless the user excludes their domain. The OAuth tokens are kept as plain files on the laptop, protected only by the file permissions of the operating system. Finally, the language model is a hosted service, so its availability and limits apply.

Future work starts with the evaluation in Section VIII. Beyond that, we plan to store several vectors for each long item and to let the domain table learn from the user's own confirmed labels. We also plan to accept very confident group suggestions automatically, while keeping them open to a weekly undo, and to optionally nest user groups under system groups. Importing past history from Google Takeout and from the browser's own history file would let a new user start with a full memory instead of an empty one. Finally, exposing the search tool through the Model Context Protocol would let other assistants query the same memory under the same privacy rules.

## X. CONCLUSION

E.C.H.O turns email, browsing, and video history into one personal memory that can be searched in plain language. It keeps all legitimate mail, but only the pages and videos the user paid attention to. It stores them on the user's laptop and prepares them for search in the background. Questions are

answered by a loop in which a language model plans the search and deterministic code performs it. This keeps the role of the model visible, bounded, and cheap. Around this loop, a wellbeing module describes how time is spent and keeps the user in charge of every personal grouping. The next version of this paper will report the measurements laid out above.

## ACKNOWLEDGMENT

We thank the Department of Computer Science and Engineering, PES University, for its support during this capstone project.

## REFERENCES

[1] V. Bush, "As we may think," *The Atlantic Monthly*, vol. 176, no. 1, pp. 101–108, Jul. 1945.

[2] S. Dumais, E. Cutrell, J. J. Cadiz, G. Jancke, R. Sarin, and D. C. Robbins, "Stuff I've seen: A system for personal information retrieval and re-use," in *Proc. 26th Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Toronto, Canada, 2003, pp. 72–79.

[3] J. Gemmell, G. Bell, and R. Lueder, "MyLifeBits: A personal database for everything," *Commun. ACM*, vol. 49, no. 1, pp. 88–95, Jan. 2006.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] LangChain, "LangGraph," GitHub repository. [Online]. Available: https://github.com/langchain-ai/langgraph (accessed Oct. 21, 2026).

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.

[8] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.

[9] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Boston, MA, USA, 2009, pp. 758–759.

[10] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[11] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 5776–5788.

[12] P. Bajaj *et al.*, "MS MARCO: A human generated machine reading comprehension dataset," arXiv:1611.09268, 2016.

[13] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178.

[14] U. Lyngs *et al.*, "Self-control in cyberspace: Applying dual systems theory to a review of digital self-control tools," in *Proc. CHI Conf. Human Factors in Computing Systems (CHI '19)*, Glasgow, UK, 2019, doi: 10.1145/3290605.3300361.

[15] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.